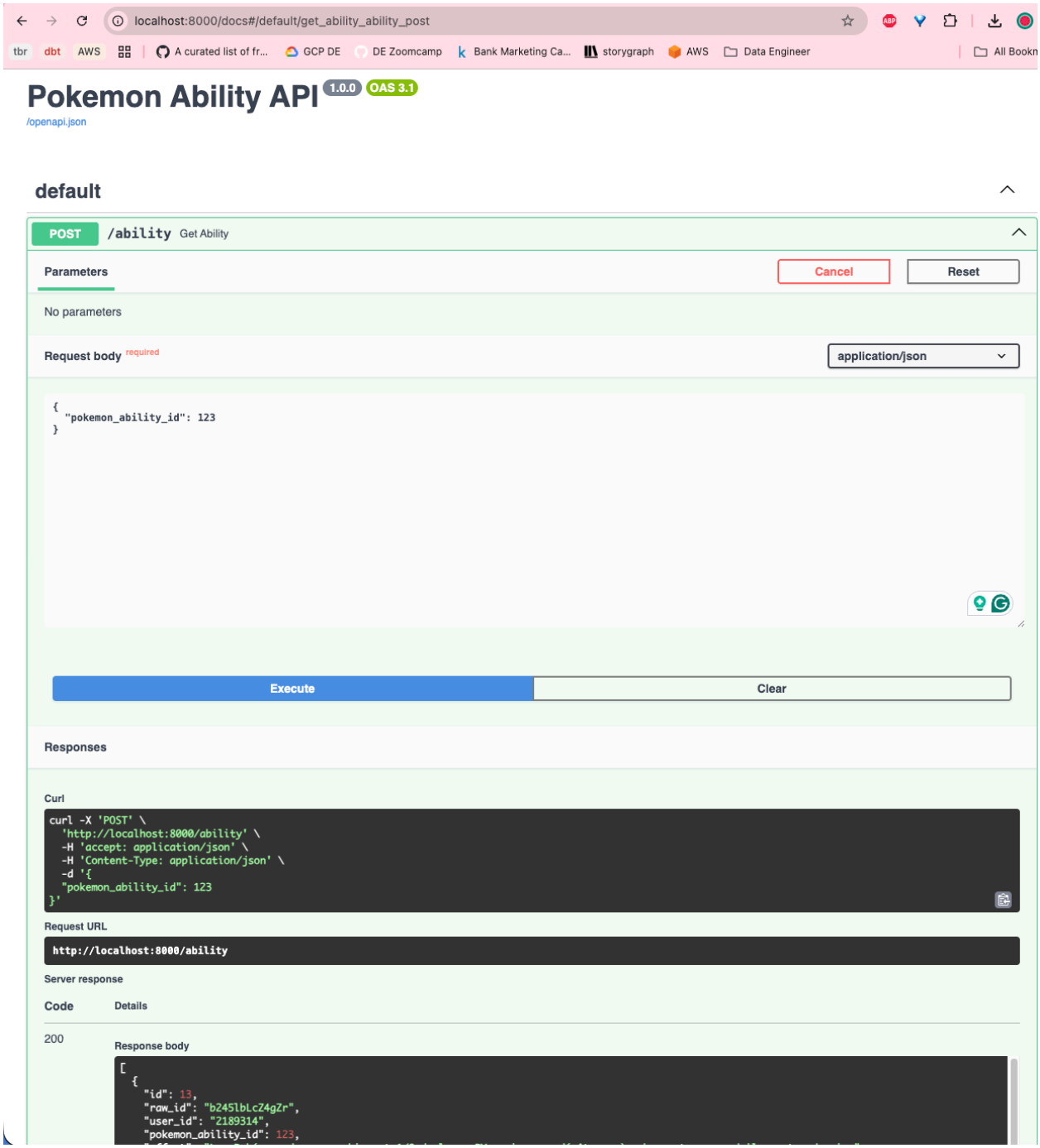
Architecture Diagram

Data Source

The source is the PokeAPI (https://pokeapi.co/api/v2/ability/{id}), consumed via the FastAPI service. Each POST /ability call fetches one ability and its effect_entries, the Pokemon list, and the metadata. FastAPI is used to handle extraction and loading to the database. The script is deployed utilizing Docker and Docker Compose for easier deployment.



Staging Area (Bronze layer)

This is an immutable temporary or persistent landing zone for the raw data stored as is from the source. We store the exact JSON response in a JSONB column (stg_ability_raw). If the parsing logic breaks, we can replay the data from the staging table instead of re-hitting the external API.

```
SELECT * FROM stg_ability_raw LIMIT 100
```

Search Results | Export | Cost: 7ms | 1 | Total 4

load_i bigint	ability_id integer	raw_payload jsonb	loaded_at timestamp with time
2	1	{"id": 1, "name": "stench", "na	2026-04-20 14:48:08.98797
3	150	{"id": 150, "name": "imposter"	2026-04-20 14:48:49.13311
4	123	{"id": 123, "name": "bad-drea	2026-04-20 14:49:03.27352
5	123	{"id": 123, "name": "bad-drea	2026-04-20 14:58:11.11269

stg_ability_raw table

The staging table is immutable, meaning it will always append no matter what. If we hit the API with ability_id 123 twice, the record would append and the freshness can be shown by the loaded_at column.

In this layer, we use FastAPI to receives the POST request and triggers the ingestion to PostgreSQL. The httpx is used instead of request to do an async HTTP client that can calls the PokeAPI.

Operational Data Store (Silver layer)

This is the normalized, clean, and current state database that becomes the source of the operational systems and applications. It is optimized for transactional row-based operations (OLTP). This layer can guarantee data integrity. By normalizing the data, we prevent duplication and ensure that if an application queries the database for a Pokemon ability, it would get a clean and structured response. In this layer, there's no denormalized tables as those belong to the Data Mart layer.

In this project, there are 2 ODS tables.

ods_abilities	ods_effect_entries
raw_id TEXT PK	entry_id SERIAL PK
user_id TEXT	effect TEXT
ability_id INT	short_effect TEXT
updated_at TIMESTAMP	language TEXT
	is_current BOOLEAN

```
SELECT * FROM ods_abilities LIMIT 100
```

Search Results | Export | Cost: 25ms

1 | Total 3

raw_id text	user_id text	ability_id integer	updated_at timestamp with time
cHTpmMKOVo8I2	9920185	1	2026-04-20 14:48:08.98797
KzZS7d19Qc2Ux	1874092	150	2026-04-20 14:48:49.13311
e0TL9MnO97FZ6	9579887	123	2026-04-20 14:49:03.27352

ods_abilities table

```
SELECT * FROM ods_effect_entries LIMIT 100
```

Search Results | Export | Cost: 4ms | 1 | Total 9

id bigint	raw_id text	user_id text	pokemon_ability_id integer	effect text	language jsonb	short_effect text	poker
4	cHTpmMKOVo8I2	9920185	1	Les capacités offensives de ce	{"url": "https://pokeapi.co/api/	A 10% de chances d'apeurer)	["gloom
5	cHTpmMKOVo8I2	9920185	1	Attacken die Schaden verursa	{"url": "https://pokeapi.co/api/	Mit jedem Treffer besteht eine	["gloom
6	cHTpmMKOVo8I2	9920185	1	This Pokémon's damaging mo	{"url": "https://pokeapi.co/api/	Has a 10% chance of making	["gloom
7	KzZS7d19Qc2Ux	1874092	150	Ce Pokémon se transforme en	{"url": "https://pokeapi.co/api/	Se transforme en entrant au c	["ditto"]
8	KzZS7d19Qc2Ux	1874092	150	Pokémon mit dieser Fähigkeit	{"url": "https://pokeapi.co/api/	Verwandelt sich beim Betreter	["ditto"]
9	KzZS7d19Qc2Ux	1874092	150	This Pokémon transforms into	{"url": "https://pokeapi.co/api/	Transforms upon entering batt	["ditto"]
10	e0TL9MnO97FZ6	9579887	123	Les Pokémon adverses subiss	{"url": "https://pokeapi.co/api/	Inflige des dégâts aux adversa	["darkra
11	e0TL9MnO97FZ6	9579887	123	Gegner erleiden am Ende jede	{"url": "https://pokeapi.co/api/	Fügt schlafenden Gegnern jed	["darkra
12	e0TL9MnO97FZ6	9579887	123	Opposing Pokémon take 1/8 o	{"url": "https://pokeapi.co/api/	Damages sleeping opponents	["darkra

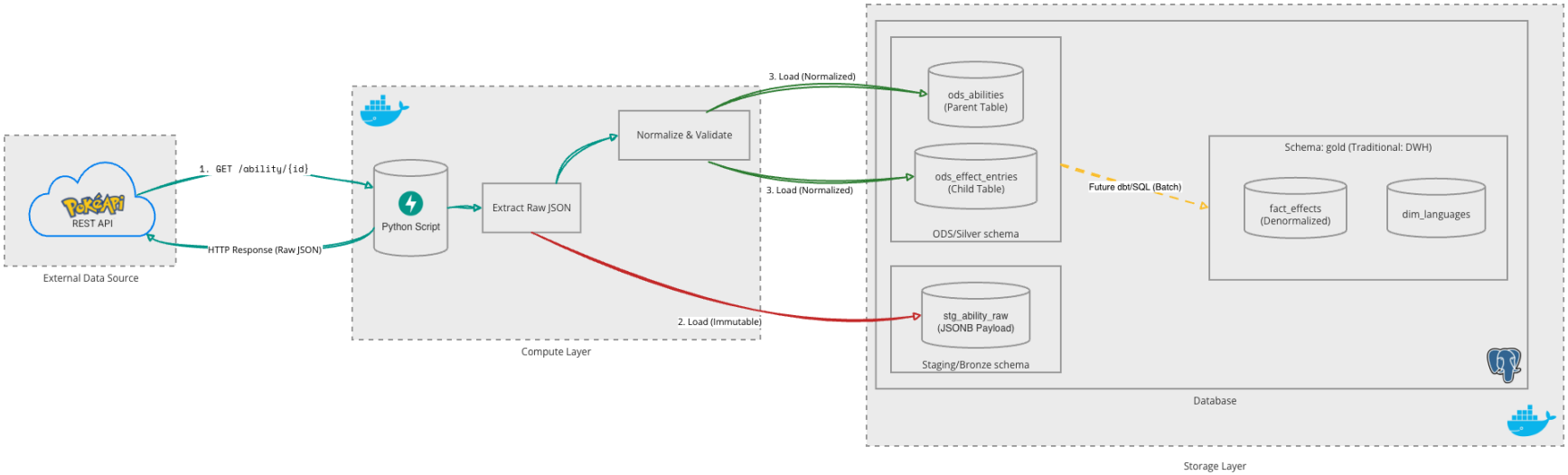
ods_effect_entries table

The column updated_at TIMESTAMP on ods_abilities can allow downstream jobs to perform incremental loads and ensure that the data is the latest data (freshness). The deduplication key used is ability_id to ensure that the data is idempotent. For example, if the ability_id 1 has 3 rows of effect language, how many times we hit the ability_id 1, in the ods_effect_entries table, the row count for ability_id would remain 3 unless there's an update in the source.

Data Warehouse (Gold layer)

This part is a denormalized star schema build using a tool like dbt for reporting or dashboard. Usually optimized for analytical queries (OLAP). It is separated from the transactional ODS to do heavy analytical reads. We can use DuckDB or any cloud data warehouse to do this. Currently, most modern data stacks utilize a data lakehouse where we can store an open table format in a data lake while directly querying the data from a data warehouse.

Diagram


REST API
External Data Source
1. GET /ability/{id}
HTTP Response (Raw JSON)
Python Script
Extract Raw JSON
Normalize & Validate
Compute Layer
3. Load (Normalized)
3. Load (Normalized)
2. Load (Immutable)
ods_abilities
(Parent Table)
ods_effect_entries
(Child Table)
ODS/Silver schema
stg_ability_raw
(JSONB Payload)
Staging/Bronze schema
Future dbt/SQL (Batch)
Schema: gold (Traditional: DWH)
fact_effects
(Denormalized)
dim_languages
Database
Storage Layer